

Módulo Financiamiento — Documentación Completa para Desarrolladores

Versión: 1.0

Módulo: Modules\Financiamiento

Framework: Laravel (Eloquent ORM)

Audiencia: Desarrolladores sin experiencia en sistemas contables/financieros

Tabla de Contenidos

1. ¿Qué es este módulo?
2. Conceptos Básicos de Contabilidad
3. Entidades del Sistema
4. Diagrama de Relaciones
5. Flujos de Negocio
6. Casos de Uso Prácticos
7. Glosario de Términos

1. ¿Qué es este módulo?

Este módulo es un **sistema de gestión financiera** integrado dentro de una aplicación Laravel modular. Su propósito es:

-  Registrar **facturas** a clientes (ingresos)
-  Registrar **gastos** de la empresa (egresos)
-  Controlar **pagos** recibidos de clientes
-  Gestionar **productos/servicios** que se facturan
-  Mantener **diarios contables** (cuentas bancarias, caja, etc.)
-  Categorizar gastos para análisis posterior

 **Analogía simple:** Imagina que este módulo es como el “**Excel de finanzas**” de la empresa, pero con validaciones, relaciones entre datos y auditoría automática.

2. Conceptos Básicos de Contabilidad

Antes de entender el código, necesitas entender estos conceptos. No te preocupes, son más simples de lo que parecen.

2.1 ¿Qué es una Factura (Invoice)?

Una **factura** es un documento que le envías a un cliente diciendo: “*Me debes X dinero por estos productos/servicios*”.

Campo	Significado
invoice_date	Fecha en que se emitió la factura
total_amount	Total a pagar (suma de todas las líneas)

Table 1 – continued

Campo	Significado
status	Estado: ¿Borrador? ¿Enviada? ¿Pagada?
partner_id	¿A quién le facturaste? (el cliente)
lines	Detalle de cada producto/servicio facturado

 **Ejemplo real:** Le facturas a “Pepito S.A.” 3 licencias de software a \$100 cada una. La factura tiene `total_amount = $300` y 3 líneas.

2.2 ¿Qué es una Línea de Factura (InvoiceLine)?

Cada factura puede tener **muchos productos**. Cada producto en la factura es una “línea”.

```
Factura #001
├─ Línea 1: 2 x Mouse ($25) = $50
├─ Línea 2: 1 x Teclado ($80) = $80
└─ Total: $130
```

Campo	Significado
quantity	Cantidad vendida
unit_price	Precio por unidad
subtotal	$quantity \times unit_price$
product_id	Qué producto es

2.3 ¿Qué es un Pago (Payment)?

Un pago es cuando el cliente **te da el dinero** para saldar una factura.

 **¡Importante!** Una factura puede tener **varios pagos** (ej: pago parcial en efectivo + transferencia).

Campo	Significado
amount	Cuánto pagó
payment_date	Cuándo pagó
journal_id	¿En qué cuenta/caja entró el dinero?
invoice_id	¿Qué factura está pagando?

2.4 ¿Qué es un Diario (Journal)?

Un **diario** es una **cuenta o caja** donde entra/sale el dinero.

Tipo de Diario	Ejemplo real
CASH	Caja fuerte de la tienda
BANK	Cuenta bancaria del Banco Santander
CREDIT_CARD	Terminal de tarjeta de crédito
WALLET	Billetera digital (PayPal, MercadoPago)

Campo	Significado
balance	Cuánto dinero hay actualmente en esa cuenta
currency	Moneda (USD, EUR, MXN, etc.)
titular	Nombre del titular de la cuenta
type	Tipo de diario (enum)

 **Ejemplo:** Tienes un diario “Caja Principal” tipo CASH con balance \$5,000. Cuando recibes un pago en efectivo, el balance sube.

2.5 ¿Qué es un Gasto (Expense)?

Un **gasto** es dinero que la empresa **gasta** (no recibe). Ejemplos: alquiler, luz, internet, salarios.

Campo	Significado
amount	Cuánto gastaste
expense_date	Cuándo gastaste
expense_category_id	¿En qué categoría? (ej: “Servicios”, “Materiales”)
journal_id	¿De qué cuenta salió el dinero?
partner_id	¿A quién le pagaste? (proveedor)

 **Ejemplo:** Pagaste \$500 de luz el 15 de junio. El dinero salió de tu “Cuenta Bancaria” (Journal tipo BANK) y la categoría es “Servicios Públicos”.

2.6 ¿Qué es una Categoría de Gasto (ExpenseCategory)?

Sirve para **agrupar** gastos y poder ver reportes del tipo: “¿Cuánto gastamos en servicios este mes?”

Campo	Significado
name	Nombre de la categoría
is_active	¿Está disponible para usar?

2.7 ¿Qué es un Producto (Product)?

Es cualquier cosa que puedas **vender** y poner en una factura.

Campo	Significado
code	Código SKU o identificador interno
name	Nombre del producto
type	Tipo: SERVICE (servicio), GOODS (bien físico), SUBSCRIPTION (suscripción)
price	Precio base de venta
is_active	¿Está disponible para facturar?

2.8 ¿Qué es un Partner?

Un **Partner** es una entidad externa del módulo Core. Representa a:

- **Clientes** (a quienes les facturas)
- **Proveedores** (a quienes les pagas gastos)
- **Ambos** (alguien puede ser cliente y proveedor)

 Es una relación **polimórfica** en el sentido de negocio: un Partner puede estar tanto en Invoice como en Expense.

3. Entidades del Sistema

3.1 Invoice (Factura)

```
class Invoice extends Model
{
    use SoftDeletes; // ← No borra físicamente, marca como "eliminado"

    protected $fillable = [
        'partner_id',    // FK → Partner (el cliente)
        'memo',          // Nota corta interna
        'invoice_date',  // Fecha de emisión
        'total_amount',  // Total de la factura
        'status',        // Enum: DRAFT, SENT, PAID, OVERDUE, CANCELLED
        'notes'          // Notas adicionales
    ];
}
```

Relaciones:

- `partner()` → Pertenece a un Partner (cliente)
- `lines()` → Tiene muchas InvoiceLine (detalle)
- `payments()` → Tiene muchos Payment (pagos recibidos)

3.2 InvoiceLine (Línea de Factura)

```
class InvoiceLine extends Model
{
    protected $fillable = [
        'invoice_id',    // FK → Invoice
        'product_id',    // FK → Product
        'quantity',      // Cantidad
        'unit_price',    // Precio unitario
        'subtotal'       // quantity × unit_price
    ];
}
```

⚠️ **No usa SoftDeletes** porque es una entidad dependiente. Si borras la factura, las líneas se borran en cascada.

3.3 Payment (Pago)

```
class Payment extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'invoice_id',      // FK → Invoice (qué factura paga)
        'journal_id',      // FK → Journal (de qué cuenta sale/entra)
        'amount',          // Monto pagado
        'payment_date',    // Fecha del pago
        'memo'             // Nota del pago
    ];
}
```

💡 **Concepto clave:** Un pago SIEMPRE está asociado a una factura. Si recibes dinero sin factura, eso sería un “ingreso directo” (no implementado aquí).

3.4 Journal (Diario / Cuenta)

```
class Journal extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'name',            // Nombre descriptivo
        'type',            // Enum: CASH, BANK, CREDIT_CARD, WALLET
        'titular',         // Nombre del titular
        'currency',        // Moneda (USD, EUR, etc.)
        'balance',         // Saldo actual
        'is_active'        // ¿Está activa?
    ];
}
```

💰 **Analogía:** Piensa en un Journal como tu “billetera” o “cuenta de banco” en la vida real. Tiene un nombre, un tipo y sabes cuánto dinero hay.

3.5 Expense (Gasto)

```
class Expense extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'expense_category_id', // FK → ExpenseCategory
    ];
}
```

```

    'journal_id',          // FK → Journal (de qué cuenta sale)
    'partner_id',         // FK → Partner (a quién pagaste)
    'amount',              // Monto gastado
    'expense_date',        // Fecha del gasto
    'description'          // Descripción del gasto
  ];
}

```

💡 **Concepto clave:** Un gasto SIEMPRE sale de un Journal (cuenta). Si pagas \$500 de luz desde tu cuenta bancaria, el `journal_id` apunta a tu diario tipo BANK.

3.6 ExpenseCategory (Categoría de Gasto)

```

class ExpenseCategory extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'name',           // Ej: "Servicios Públicos"
        'description',    // Descripción larga
        'is_active'       // ¿Disponible para usar?
    ];
}

```

🇨🇱 **Para qué sirve:** Para poder hacer reportes como “*Gastos por categoría del mes de junio*”.

3.7 Product (Producto)

```

class Product extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'code',           // SKU o código interno
        'name',           // Nombre del producto
        'type',           // Enum: SERVICE, GOODS, SUBSCRIPTION
        'price',          // Precio base
        'is_active'       // ¿Disponible para venta?
    ];
}

```

🛒 Ejemplos:

- SERVICE: “Consultoría de 1 hora” (\$50)
- GOODS: “Laptop Dell XPS” (\$1,200)
- SUBSCRIPTION: “Plan Mensual SaaS” (\$29/mes)

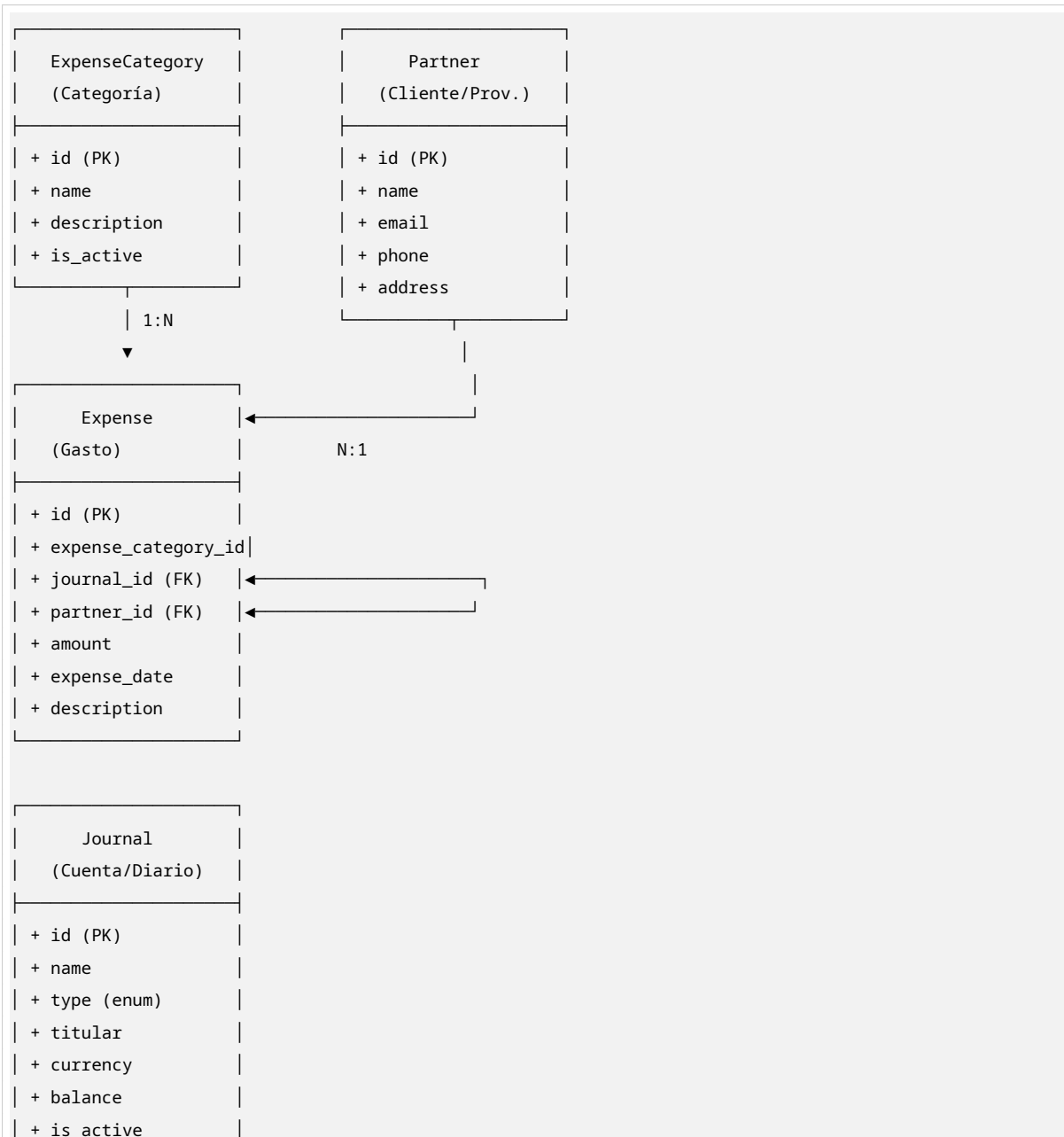
3.8 Partner (Entidad Externa — Módulo Core)

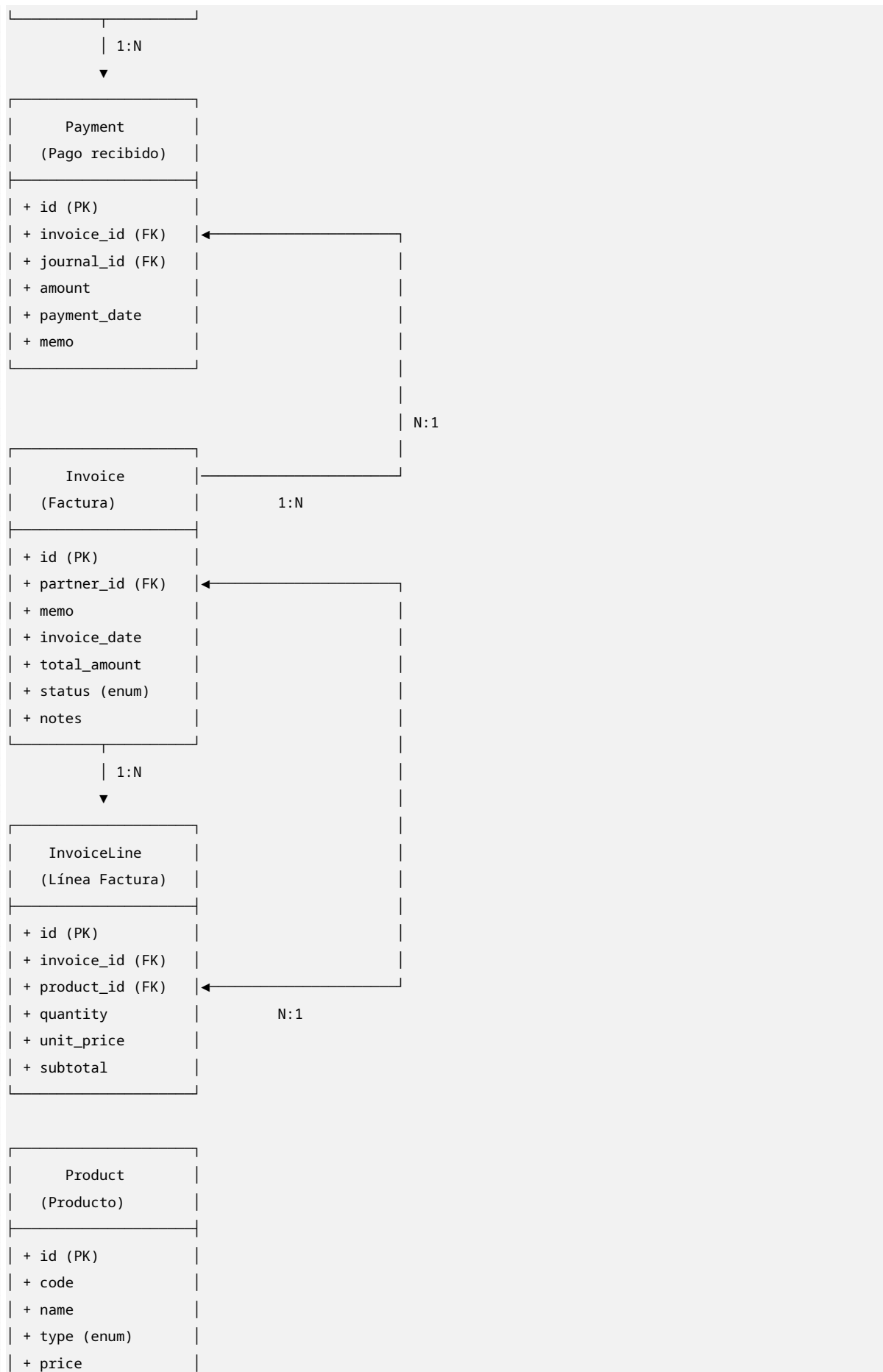
```
// Ubicado en: Modules\Core\App\Models\Partner
class Partner extends Model
{
    // Datos de contacto, dirección, etc.
}
```

Este módulo no lo define, pero lo **usa**. Un Partner puede ser:

- **Cliente** → aparece en Invoice
- **Proveedor** → aparece en Expense
- **Ambos** → aparece en ambos

4. Diagrama de Relaciones






```
| + is_active |
```

5. Flujos de Negocio

5.1 Flujo Completo: Facturación y Cobro

Paso 1: Crear Productos

↳ Product ("Consultoría", \$100/hr)

Paso 2: Crear un Partner (Cliente)

↳ Partner ("Acme Corp", contacto@acme.com)

Paso 3: Crear una Factura

↳ Invoice (partner_id = Acme Corp, status = DRAFT)

↳ InvoiceLine (producto = Consultoría, qty = 5, unit_price = \$100)

↳ InvoiceLine (producto = Reporte, qty = 1, unit_price = \$50)

[AUTO-CALCULAR] total_amount = (5 × \$100) + (1 × \$50) = \$550

Paso 4: Enviar Factura

↳ status cambia de DRAFT → SENT

Paso 5: Cliente paga (ej: \$300 en efectivo)

↳ Payment (invoice_id = Invoice, journal_id = Caja, amount = \$300)

↳ Invoice.status = PARCIAL (o sigue SENT si no hay lógica de estados)

Paso 6: Cliente paga el resto (\$250 por transferencia)

↳ Payment (invoice_id = Invoice, journal_id = Banco, amount = \$250)

↳ Invoice.status = PAID (si total pagado == total_amount)

Paso 7: Reporte

↳ "Factura #001 de Acme Corp: \$550 → Pagada en 2 pagos"

5.2 Flujo Completo: Registro de Gasto

Paso 1: Crear Categorías de Gasto

↳ ExpenseCategory ("Servicios Públicos")

↳ ExpenseCategory ("Materiales de Oficina")

Paso 2: Crear Diarios (Cuentas)

↳ Journal ("Cuenta Bancaria Principal", tipo BANK, balance = \$10,000)

↳ Journal ("Caja Chica", tipo CASH, balance = \$500)

Paso 3: Registrar un Gasto

↳ Expense (
category_id = "Servicios Públicos",
journal_id = "Cuenta Bancaria Principal",
partner_id = "Empresa de Luz S.A.",

```
        amount = $450,
        expense_date = "2026-06-10",
        description = "Factura de luz junio 2026"
    )

    [AUTO-ACTUALIZAR] Journal.balance = $10,000 - $450 = $9,550
```

Paso 4: Reporte

```
↳ "Gastos en Servicios Públicos: $450"
↳ "Saldo en Cuenta Bancaria: $9,550"
```

6. Casos de Uso Prácticos

Caso 1: SaaS que factura suscripciones mensuales

```
// 1. Crear producto tipo suscripción
$product = Product::create([
    'code' => 'PLAN-PRO',
    'name' => 'Plan Pro Mensual',
    'type' => ProductTypeEnum::SUBSCRIPTION,
    'price' => 29.99,
    'is_active' => true
]);

// 2. Crear cliente
$cliente = Partner::create(['name' => 'Startup XYZ']);

// 3. Crear factura mensual
$invoice = Invoice::create([
    'partner_id' => $cliente->id,
    'invoice_date' => now(),
    'status' => InvoiceStatusEnum::DRAFT,
    'notes' => 'Suscripción Junio 2026'
]);

// 4. Agregar línea
InvoiceLine::create([
    'invoice_id' => $invoice->id,
    'product_id' => $product->id,
    'quantity' => 1,
    'unit_price' => $product->price,
    'subtotal' => $product->price
]);

// 5. Actualizar total
$invoice->update(['total_amount' => $product->price]);

// 6. Enviar (cambiar estado)
$invoice->update(['status' => InvoiceStatusEnum::SENT]);
```

```
// 7. Cliente paga con tarjeta
Payment::create([
    'invoice_id' => $invoice->id,
    'journal_id' => $terminalTarjeta->id, // Journal tipo CREDIT_CARD
    'amount' => $invoice->total_amount,
    'payment_date' => now(),
    'memo' => 'Pago con tarjeta terminación 4242'
]);

// 8. Marcar como pagada
$invoice->update(['status' => InvoiceStatusEnum::PAID]);
```

Caso 2: Tienda física que vende productos

```
// Cliente compra 2 camisetas y 1 gorra
$invoice = Invoice::create([
    'partner_id' => $cliente->id,
    'invoice_date' => now(),
    'status' => InvoiceStatusEnum::DRAFT
]);

// Línea 1: Camisetas
InvoiceLine::create([
    'invoice_id' => $invoice->id,
    'product_id' => $camiseta->id,
    'quantity' => 2,
    'unit_price' => 25.00,
    'subtotal' => 50.00
]);

// Línea 2: Gorra
InvoiceLine::create([
    'invoice_id' => $invoice->id,
    'product_id' => $gorra->id,
    'quantity' => 1,
    'unit_price' => 15.00,
    'subtotal' => 15.00
]);

$invoice->update(['total_amount' => 65.00]);

// Pago en efectivo
Payment::create([
    'invoice_id' => $invoice->id,
    'journal_id' => $caja->id, // Journal tipo CASH
    'amount' => 65.00,
    'payment_date' => now()
]);
```

Caso 3: Empresa que paga servicios

```
// Pagar factura de internet
$expense = Expense::create([
    'expense_category_id' => $servicios->id,
    'journal_id' => $cuentaBanco->id,
    'partner_id' => $proveedorInternet->id,
    'amount' => 89.99,
    'expense_date' => '2026-06-01',
    'description' => 'Factura de internet junio 2026'
]);

// El balance del diario debería actualizarse (esto requiere lógica adicional)
// $cuentaBanco->balance -= 89.99;
```

7. Glosario de Términos

Término	Explicación Simple
Invoice	Documento que dice “me debes plata”
InvoiceLine	Cada producto/servicio dentro de una factura
Payment	Cuando alguien te paga una factura
Journal	Una cuenta o caja donde guardas dinero
Expense	Dinero que la empresa gasta
ExpenseCategory	Grupo para clasificar gastos
Product	Cualquier cosa que vendas
Partner	Persona o empresa con la que haces negocios
SoftDeletes	No borra realmente, solo oculta (para auditoría)
Fillable	Campos que se pueden asignar masivamente
Casts	Convierte tipos de datos automáticamente
Enum	Lista fija de valores permitidos
FK	Foreign Key = Llave foránea (relación a otra tabla)
PK	Primary Key = ID único de cada registro
Balance	Cuánto dinero hay en una cuenta
Subtotal	Cantidad × Precio unitario
Total Amount	Suma de todos los subtotales de una factura
DRAFT	Estado: Factura en borrador, no enviada
SENT	Estado: Factura enviada al cliente
PAID	Estado: Factura totalmente pagada
OVERDUE	Estado: Factura vencida, no pagada a tiempo
CANCELLED	Estado: Factura anulada
CASH	Tipo de diario: Dinero en efectivo
BANK	Tipo de diario: Cuenta bancaria
CREDIT_CARD	Tipo de diario: Terminal de tarjeta
WALLET	Tipo de diario: Billetera digital
SERVICE	Tipo de producto: Servicio intangible

Table 9 – continued

Término	Explicación Simple
GOODS	Tipo de producto: Bien físico
SUBSCRIPTION	Tipo de producto: Pago recurrente
BelongsTo	Relación: “Pertenece a” (el hijo apunta al padre)
HasMany	Relación: “Tiene muchos” (el padre tiene hijos)
Memo	Nota corta interna (no aparece en la factura impresa)
Notes	Notas adicionales (pueden ser más largas)
Titular	Persona a nombre de quien está una cuenta bancaria
SKU	Stock Keeping Unit = Código interno de producto
Audit Trail	Rastro de quién hizo qué y cuándo

8. Notas Técnicas Importantes

8.1 SoftDeletes

Las entidades con **SoftDeletes** no se eliminan físicamente de la base de datos. En su lugar, se marca la columna `deleted_at` con la fecha de eliminación. Esto permite:

- Recuperar datos borrados por error
- Mantener historial para auditoría
- Evitar pérdida de información financiera crítica

⚠ InvoiceLine NO tiene SoftDeletes porque depende 100% de Invoice. Si borras la factura, las líneas deben desaparecer.

8.2 Timestamps

Todas las entidades Eloquent tienen automáticamente:

- `created_at`: Cuándo se creó el registro
- `updated_at`: Cuándo se modificó por última vez

8.3 Casts (Conversiones Automáticas)

```
protected $casts = [
    'amount' => 'decimal:2',      // Siempre 2 decimales (ej: 100.00)
    'expense_date' => 'date',      // Objeto Carbon (fecha)
    'is_active' => 'boolean',      // true/false
    'status' => InvoiceStatusEnum::class // Enum PHP 8.1+
];
```

8.4 Relaciones y Cardinalidad

Relación	Significado	Ejemplo
1:N	Uno a Muchos	Una categoría tiene muchos gastos
N:1	Muchos a Uno	Muchos gastos pertenecen a una categoría
1:1	Uno a Uno	(No usado en este módulo)
N:M	Muchos a Muchos	(No usado en este módulo)

9. Preguntas Frecuentes (FAQ)

¿Por qué InvoiceLine no tiene SoftDeletes?

Porque es una entidad dependiente. Las líneas no existen sin la factura. Si necesitas recuperar una factura borrada, restauras la factura y sus líneas se restauran en cascada (si configuraste la base de datos así).

¿Qué pasa si un cliente paga más de lo que debe?

El sistema actual no tiene lógica de “sobrepago”. Tendrías que implementar un “crédito a favor” o devolver el excedente.

¿Puedo tener una factura sin líneas?

Técnicamente sí (a nivel de base de datos), pero no tiene sentido de negocio. Deberías validar que toda factura tenga al menos una línea.

¿Qué es la diferencia entre **memo** y **notes**?

- **memo**: Nota corta, interna, para uso administrativo
- **notes**: Notas más largas, pueden ser para el cliente o para registro interno detallado

¿Por qué Payment tiene **journal_id**?

Para saber **en qué cuenta** entró el dinero. Si te pagan \$100 en efectivo, el balance de la caja sube. Si te pagan por transferencia, el balance del banco sube.

¿Y si un pago cubre varias facturas?

El modelo actual asume 1 pago = 1 factura. Para pagos que cubren múltiples facturas, necesitarías una tabla intermedia o dividir el pago en varios registros.

Documento generado para el equipo de desarrollo. Última actualización: Junio 2026